KANPIM: nine-lane FPGA spline vs CPU and CUDA GPU

This notebook compares the element-wise cubic spline

$$S(t) = ((at + b)t + c)t + d$$

across the synthesized nine-lane FPGA RTL, NumPy on CPU, and PyTorch on CUDA.

- **CPU/GPU:** measured in this Colab runtime with resident data.
- **FPGA resources:** actual Vivado 2024.1 synthesis results.
- **FPGA performance:** calculated from the clock-constrained post-synthesis timing result. This remains an estimate until placement, routing, and board measurement.

```
import time, platform
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch

print("CPU:", platform.processor() or platform.machine())
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CPU: x86_64
PyTorch: 2.11.0+cu130
CUDA available: True
GPU: Tesla T4
```

1. Actual Vivado synthesis information

Device: **XC7A100T-CSG324-1**.

The design inferred nine DSP48E1 multipliers and nine independent 16 x 64-bit coefficient memories, each mapped to one RAMB36E1. No latches were inferred.

```
fpga_resources = pd.DataFrame({
    "Resource": ["Slice LUTs", "Slice registers", "DSP48E1", "RAMB36E1"],
    "Used": [311, 371, 9, 9],
    "Available": [63400, 126800, 240, 135],
    "Utilization (%)": [0.49, 0.29, 3.75, 6.67],
})
display(fpga_resources)

fpga_facts = pd.Series({
    "Vivado": "2024.1.2",
    "Device": "xc7a100tcsg324-1",
    "Lanes": 9,
    "Data format": "signed 16-bit Q8.8, wrapping",
    "Segments/lane": 16,
    "Latency cycles": 4,
    "Initiation interval cycles": 5,
    "Clock constraint present": True,
    "Clock constraint (ns)": 10.0,
    "WNS (ns)": -0.064,
    "TNS (ns)": -0.485,
    "Setup failing endpoints": 8,
    "Hold WNS (ns)": 0.266,
})
display(fpga_facts)
```

	Resource	Used	Available	Utilization (%)
0	Slice LUTs	311	63400	0.49
1	Slice registers	371	126800	0.29
2	DSP48E1	9	240	3.75
3	RAMB36E1	9	135	6.67

	0
Vivado	2024.1.2
Device	xc7a100tcsg324-1
Lanes	9
Data format	signed 16-bit Q8.8, wrapping
Segments/lane	16
Latency cycles	4
Initiation interval cycles	5
Clock constraint present	True
Clock constraint (ns)	10.0
WNS (ns)	-0.064
TNS (ns)	-0.485
Setup failing endpoints	8
Hold WNS (ns)	0.266

dtype: object

Next steps: Generate code with `fpga_resources` New interactive sheet

Report interpretation

- All nine multipliers mapped to DSP48E1 blocks.
- All nine coefficient banks mapped to RAMB36E1 blocks.
- The 73 synthesis warnings mainly repeat that four upper bits of each 16-bit input lane are unused. For inputs clamped to [0,16), the design uses fractional bits 7:0 and segment bits 11:8, so bits 15:12 have no functional role.
- The constrained run has no unconstrained internal endpoints.
- Setup timing narrowly fails at 100 MHz: WNS is -0.064 ns, TNS is -0.485 ns, and 8 of 1784 endpoints fail.
- Hold timing passes with WHS = +0.266 ns and THS = 0.
- Asynchronous resets prevent registers from being absorbed into DSP blocks and remain an optimization target.
- Therefore, synthesis and mapping succeeded, but the design does not formally close timing at 100 MHz.

2. Clock-constrained post-synthesis estimate

The 10 ns constraint missed by 0.064 ns, giving a post-synthesis critical-path estimate of 10.064 ns and an estimated maximum frequency of 99.36 MHz. This is not a routed or board-measured frequency.

```
FPGA_CONSTRAINT_NS = 10.0
FPGA_WNS_NS = -0.064
FPGA_CRITICAL_PATH_NS_EST = FPGA_CONSTRAINT_NS - FPGA_WNS_NS
FPGA_CLOCK_MHZ_PROJECTED = 1000.0 / FPGA_CRITICAL_PATH_NS_EST
FPGA_LANES = 9
FPGA_LATENCY_CYCLES = 4
FPGA_II_CYCLES = 5

fpga_latency_us = FPGA_LATENCY_CYCLES / FPGA_CLOCK_MHZ_PROJECTED
fpga_throughput = FPGA_LANES * FPGA_CLOCK_MHZ_PROJECTED * 1e6 / FPGA_II_CYCLES
display(pd.Series({
    "Evidence": "clock-constrained post-synthesis estimate; setup timing misses 100 MHz",
    "Estimated critical path (ns)": FPGA_CRITICAL_PATH_NS_EST,
    "Estimated maximum clock (MHz)": FPGA_CLOCK_MHZ_PROJECTED,
    "Transaction latency (us)": fpga_latency_us,
    "Throughput (elements/s)": fpga_throughput,
}))
```

	0
Evidence	clock-constrained post-synthesis estimate; set...
Estimated critical path (ns)	10.064
Estimated maximum clock (MHz)	99.36407
Transaction latency (us)	0.040256
Throughput (elements/s)	178855325.914149

dtype: object

3. Common software workload

CPU and GPU use identical FP32 coefficients and inputs. The RTL is Q8.8, so this is a performance comparison, not a bit-exact precision comparison.

```
LANES, NUM_SEGMENTS = 9, 16
rng = np.random.default_rng(7)
coeff_np = rng.standard_normal((NUM_SEGMENTS, 4)).astype(np.float32)

def spline_cpu(x, coeff):
    idx = np.clip(np.floor(x).astype(np.int64), 0, coeff.shape[0]-1)
    t = x - idx.astype(np.float32)
    c = coeff[idx]
    return ((c[...,0]*t + c[...,1])*t + c[...,2])*t + c[...,3]

def spline_gpu(x, coeff):
    idx = torch.floor(x).long().clamp_(0, coeff.shape[0]-1)
    t = x - idx.to(x.dtype)
    c = coeff[idx]
    return ((c[...,0]*t + c[...,1])*t + c[...,2])*t + c[...,3]

coeff_gpu = torch.from_numpy(coeff_np).cuda() if torch.cuda.is_available() else None
x_check = (rng.random((8, LANES), dtype=np.float32)*NUM_SEGMENTS).astype(np.float32)
if torch.cuda.is_available():
    err = np.max(np.abs(spline_cpu(x_check, coeff_np) -
                        spline_gpu(torch.from_numpy(x_check).cuda(), coeff_gpu).cpu().numpy()))
    print("Maximum CPU/GPU FP32 difference:", err)
```

```
Maximum CPU/GPU FP32 difference: 0.0
```

4. Benchmark CPU and GPU

Median latency is reported. GPU timing uses CUDA events and synchronization. Transfers are excluded for both GPU and FPGA.

```
BATCHES = [1, 4, 16, 64, 256, 1024, 4096, 16384]

def bench_cpu(n, warmup=20, repeats=100):
    x = (rng.random((n,LANES), dtype=np.float32)*NUM_SEGMENTS).astype(np.float32)
    for _ in range(warmup): spline_cpu(x, coeff_np)
    samples=[]
    for _ in range(repeats):
        t0=time.perf_counter_ns(); spline_cpu(x, coeff_np); samples.append(time.perf_counter_ns()-t0)
    us=float(np.median(samples)/1000)
    return ["CPU NumPy", n, n*LANES, us, n*LANES/(us*1e-6)]

def bench_gpu(n, warmup=50, repeats=200):
    if not torch.cuda.is_available(): return ["CUDA unavailable",n,n*LANES,np.nan,np.nan]
    x=torch.rand(n,LANES,device="cuda")*NUM_SEGMENTS
    for _ in range(warmup): spline_gpu(x,coeff_gpu)
    torch.cuda.synchronize()
    start,end=torch.cuda.Event(True),torch.cuda.Event(True); samples=[]
    for _ in range(repeats):
        start.record(); spline_gpu(x,coeff_gpu); end.record(); end.synchronize()
        samples.append(start.elapsed_time(end)*1000)
    us=float(np.median(samples))
    return ["CUDA GPU", n, n*LANES, us, n*LANES/(us*1e-6)]

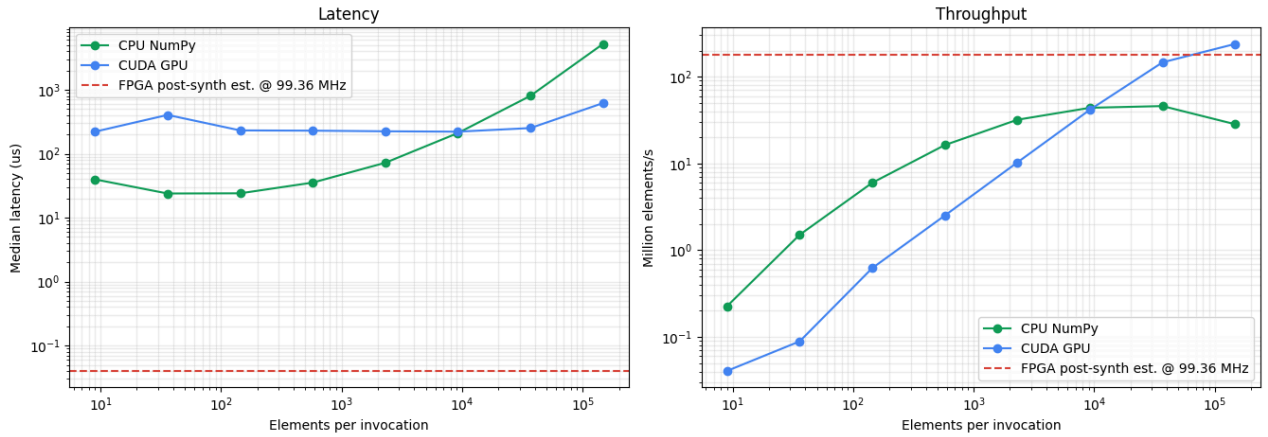
cols=["platform","transactions","elements","median_us","throughput_elements_s"]
cpu=pd.DataFrame([bench_cpu(n) for n in BATCHES],columns=cols)
gpu=pd.DataFrame([bench_gpu(n) for n in BATCHES],columns=cols)
results=pd.concat([cpu,gpu],ignore_index=True)
display(results)
```

	platform	transactions	elements	median_us	throughput_elements_s
0	CPU NumPy	1	9	39.702000	2.266888e+05
1	CPU NumPy	4	36	23.889500	1.506938e+06
2	CPU NumPy	16	144	24.047000	5.988273e+06
3	CPU NumPy	64	576	35.361500	1.628890e+07
4	CPU NumPy	256	2304	72.203500	3.190981e+07
5	CPU NumPy	1024	9216	210.283500	4.382655e+07
6	CPU NumPy	4096	36864	803.879500	4.585762e+07
7	CPU NumPy	16384	147456	5203.411500	2.833833e+07
8	CUDA GPU	1	9	221.215993	4.068422e+04
9	CUDA GPU	4	36	404.367998	8.902782e+04
10	CUDA GPU	16	144	231.455997	6.221485e+05
11	CUDA GPU	64	576	229.456000	2.510285e+06
12	CUDA GPU	256	2304	224.192001	1.027691e+07
13	CUDA GPU	1024	9216	221.247993	4.165462e+07
14	CUDA GPU	4096	36864	251.904011	1.463415e+08
15	CUDA GPU	16384	147456	618.463993	2.384229e+08

Next steps: Generate code with `results` New interactive sheet

5. Comparison plots

```
fig,ax=plt.subplots(1,2,figsize=(13.5,4.8))
colors={"CPU NumPy":"#0F9D58","CUDA GPU":"#4285F4"}
for name,g in results.dropna().groupby("platform"):
    ax[0].plot(g.elements,g.median_us,"o-",label=name,color=colors.get(name))
    ax[1].plot(g.elements,g.throughput_elements_s/1e6,"o-",label=name,color=colors.get(name))
ax[0].axhline(fpga_latency_us,color="#DB4437",ls="--",label=f"FPGA post-synth est. @ {FPGA_CLOCK_MHZ_PROJECTED:.2f} MHz")
ax[1].axhline(fpga_throughput/1e6,color="#DB4437",ls="--",label=f"FPGA post-synth est. @ {FPGA_CLOCK_MHZ_PROJECTED:.2f} MHz"
for a in ax:
    a.set_xscale("log"); a.set_yscale("log"); a.grid(True,which="both",alpha=.25); a.legend()
ax[0].set(title="Latency",xlabel="Elements per invocation",ylabel="Median latency (us)")
ax[1].set(title="Throughput",xlabel="Elements per invocation",ylabel="Million elements/s")
plt.tight_layout(); plt.show()
```



6. Export summary

```
rows=[{
    "platform":"FPGA nine-lane spline",
    "format":"Q8.8",
    "latency_us":fpga_latency_us,
    "throughput_elements_s":fpga_throughput,
    "evidence":f"clock-constrained post-synthesis estimate at {FPGA_CLOCK_MHZ_PROJECTED:.2f} MHz; not routed"
}]
for name,g in results.dropna().groupby("platform"):
    best=g.loc[g.throughput_elements_s.idxmax()]
    rows.append({"platform":name,"format":"FP32","latency_us":best.median_us,
                 "throughput_elements_s":best.throughput_elements_s,
                 "evidence":"measured in this Colab runtime; transfers excluded"})
summary=pd.DataFrame(rows)
display(summary)
summary.to_csv("kanpim_fpga_cpu_gpu_summary.csv",index=False)
results.to_csv("kanpim_cpu_gpu_raw.csv",index=False)
fpga_resources.to_csv("kanpim_fpga_resources.csv",index=False)
```

	platform	format	latency_us	throughput_elements_s	evidence
0	FPGA nine-lane spline	Q8.8	0.040256	1.788553e+08	clock-constrained post-synthesis estimate at 9...
1	CPU NumPy	FP32	803.879500	4.585762e+07	measured in this Colab runtime; transfers excl...
2	CUDA GPU	FP32	618.463993	2.384229e+08	measured in this Colab runtime; transfers excl...

Next steps: Generate code with summary | New interactive sheet

```
from google.colab import files
import zipfile

output_files = [
    "kanpim_fpga_cpu_gpu_summary.csv",
    "kanpim_cpu_gpu_raw.csv",
    "kanpim_fpga_resources.csv",
]
```